

LAPORAN PRATIKUM
STRUKTUR DATA
“DOUBLE LINKED LIST“

PEKAN KE-6

disusun Oleh:

Aisyah Najmiatul Fauziah

NIM 2511533030

Dosen Pengampu: DR. WAHYUDI, S.T, M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis ucapkan ke hadirat Allah SWT karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan topik “Double Linked List” ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai salah satu tugas praktikum pada mata kuliah Struktur Data.

Dalam praktikum ini, penulis mempelajari bagaimana konsep Double Linked List diterapkan dalam pembuatan program sederhana berupa playlist lagu. Program tersebut mampu menambah lagu, menghapus lagu, menampilkan data secara maju maupun mundur, serta mencari lagu berdasarkan judul.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menambah pemahaman mengenai struktur data Double Linked List.

Padang, 14 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Praktikum “NodeDLL_2511533030”	2
2.2 Praktikum “PenelusuranDLL_2511533030”	2
2.3 Praktikum “InsertDLL_2511533030”	2
2.4 Pratikum “HapusDLL_2511533030.....	3
BAB III.....	3
KESIMPULAN.....	3
3.1 Kesimpulan.....	3
DAFTAR PUSTAKA.....	4

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang digunakan untuk mengatur dan mengelola data agar lebih efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Linked List terbagi menjadi beberapa jenis, salah satunya adalah Double Linked List.

Double Linked List adalah struktur data yang setiap node-nya memiliki dua pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, data dapat diakses dari dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan.

Pada praktikum ini, konsep Double Linked List diterapkan dalam pembuatan program playlist lagu sederhana. Program ini digunakan untuk memahami cara kerja pointer next dan prev dalam menghubungkan node-node sehingga data dapat dikelola dengan lebih fleksibel.

1.2 Tujuan Pratikum

1. Memahami konsep dasar Double Linked List.
2. Memahami penggunaan pointer next dan prev pada node.
3. Melatih kemampuan membuat program menggunakan bahasa Java.
4. Mengimplementasikan operasi dasar Double Linked List seperti menambah, menghapus, menampilkan, dan mencari data.

1.3 Manfaat Pratikum

1. Menambah pemahaman mengenai struktur data Double Linked List.
2. Melatih logika pemrograman dalam pengelolaan data.
3. Membantu memahami hubungan antar node menggunakan pointer.
4. Melatih kemampuan membuat program yang terstruktur dan rapi.

BAB II PEMBAHASAN

2.1 Pratikum “NodeDLL_2511533030”

Langkah Kerja :

1. Buat projek, Package, dan Class baru
2. Buat kode berikut

```
package pekan6_2511533030;

public class NodeDLL_2511533030 {
    // mendefinisikan kelas Node
    int data_3030; // data
    NodeDLL_2511533030 next_3030; // pointer ke next node
    NodeDLL_2511533030 prev_3030; // pointer ke previous node

    // konstruktor
    public NodeDLL_2511533030(int data) {
        this.data_3030 = data;
        this.next_3030 = null;
        this.prev_3030 = null;
    }
}
```

Gambar 2.1 kode program “NodeDLL_2511533030”

Penjelasan Kode:

2.1.1 Penjelasan dan Deklarasi Class

- Bagian ini digunakan untuk menentukan package tempat program disimpan, yaitu **pekan6_2511533030**
- Kemudian dibuat kelas bernama **NodeDLL_2511533030** yang berfungsi sebagai node pada struktur data Double Linked List.

```
• package pekan6_2511533030;
•
• public class NodeDLL_2511533030 {
```

2.1.2 Atribut Node

- Bagian ini berisi atribut atau variabel yang dimiliki setiap node
- Karena memiliki dua pointer (next dan prev), maka node dapat diakses dari dua arah, yaitu maju dan mundur

```
// mendefinisikan kelas Node
int data_3030; // data
NodeDLL_2511533030 next_3030; // pointer ke next node
NodeDLL_2511533030 prev_3030; // pointer ke previous node
```

2.1.3 Constructor Node

- Constructor digunakan untuk membuat node baru dan menginisialisasi nilainya.

```
// konstruktor
public NodeDLL_2511533030(int data) {
    this.data_3030 = data;
    this.next_3030 = null;
    this.prev_3030 = null;
}
}
```

2.2 Pratikum “PenelusuranDLL_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat kode berikut

```
package pekan6_2511533030;

public class PenelusuranDLL_2511533030 {
    // fungsi penelusuran maju
    static void forwardTraversal (NodeDLL_2511533030 head_3030) {
        // memulai penelusuran dari head
        NodeDLL_2511533030 curr_3030 = head_3030;
        // lanjutkan sampai akhir
        while (curr_3030 != null) {
            // print data
            System.out.print(curr_3030.data_3030 + " <-> ");
            // pindah ke node berikutnya
            curr_3030 = curr_3030.next_3030;
        }
        // print spasi
        System.out.println();
    }
    // fungsi penelusuran mundur
    static void backwardTraversal(NodeDLL_2511533030 tail_3030) {
        // mulai dari akhir
        NodeDLL_2511533030 curr_3030 = tail_3030;
        // lanjut sampai head
        while (curr_3030 != null) {
            // cetak data
            System.out.print(curr_3030.data_3030 + " <-> ");
            // pindah ke node sebelumnya
            curr_3030 = curr_3030.prev_3030;
        }
        // cetak spasi
        System.out.println();
    }
    public static void main(String[] args) {
        // cetak DLL
        NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(1);
        NodeDLL_2511533030 second_3030 = new NodeDLL_2511533030(2);
        NodeDLL_2511533030 third_3030 = new NodeDLL_2511533030(3);

        head_3030.next_3030 = second_3030;
        second_3030.prev_3030 = head_3030;
        second_3030.next_3030 = third_3030;
        third_3030.prev_3030 = second_3030;

        System.out.println("Penelusuran maju:");
        forwardTraversal(head_3030);

        System.out.println("Penelusuran mundur:");
        backwardTraversal(third_3030);
    }
}
```

```
}
```

Gambar 2.2 kode program “PenelusuranDLL_2511533030”

3. Didapatkan output program sebagai berikut

```
<terminated> PenelusuranDLL
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 2.2 output program “PenelusuranDLL_2511533030”

Penjelasan Kode :

2.2.1 Deklarasi Class

- Bagian ini menunjukkan bahwa program berada di package **pekan6_2511533030** dan membuat kelas bernama **PenelusuranDLL_2511533030**
- Kelas ini digunakan untuk melakukan penelusuran (traversal) pada Double Linked List, baik secara maju maupun mundur.

```
• package pekan6_2511533030;
•
• public class PenelusuranDLL_2511533030 {
```

2.2.2 Fungsi Penelusuran Maju

- Method **forwardTraversal()** digunakan untuk menampilkan data dari depan ke belakang.
- Variabel **curr_3030** digunakan sebagai pointer bantu yang dimulai dari head.
- Perulangan berjalan selama node masih ada.
- Pointer berpindah ke node berikutnya menggunakan `next_3030`.
- Jadi, traversal maju bergerak

```
// fungsi penelusuran maju]
static void forwardTraversal (NodeDLL_2511533030 head_3030) {
    // memulai penelusuran dari head
    NodeDLL_2511533030 curr_3030 = head_3030;
    // lanjutkan sampai akhir
    while (curr_3030 != null) {
        // print data
        System.out.print(curr_3030.data_3030 + " <-> ");
        // pindah ke node berikutnya
        curr_3030 = curr_3030.next_3030;
    }
    // print spasi
    System.out.println();
}
```

2.2.3 Fungsi Penelusuran Mundur

- Method **backwardTraversal()** digunakan untuk menampilkan data dari belakang ke depan.
- Pointer bantu dimulai dari node terakhir (tail).

- Perulangan berjalan selama node masih ada.
- Pointer bergerak ke node sebelumnya menggunakan prev_3030.
- inilah kelebihan Double Linked List karena dapat ditelusuri dari dua arah.

```
// fungsi penelusuran mundur
static void backwardTraversal(NodeDLL_2511533030 tail_3030) {
    // mulai dari akhir
    NodeDLL_2511533030 curr_3030 = tail_3030;
    // lanjut sampai head
    while (curr_3030 != null) {
        // cetak data
        System.out.print(curr_3030.data_3030 + " <-> ");
        // pindah ke node sebelumnya
        curr_3030 = curr_3030.prev_3030;
    }
}
```

2.2.4 Pembuatan Node pada Main Program

- Bagian ini membuat 3 node dengan data

```
// cetak DLL
NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(1);
NodeDLL_2511533030 second_3030 = new NodeDLL_2511533030(2);
NodeDLL_2511533030 third_3030 = new NodeDLL_2511533030(3);
```

2.2.5 Menghubungkan Node

- Bagian ini menghubungkan node menggunakan pointer next dan prev.

```
head_3030.next_3030 = second_3030;
second_3030.prev_3030 = head_3030;
second_3030.next_3030 = third_3030;
third_3030.prev_3030 = second_3030;
```

2.2.6 Menjalankan Traversal

- Menampilkan data dari depan ke belakang.

```
System.out.println("Penelusuran maju:");
forwardTraversal(head_3030);
```

- Menampilkan data dari belakang ke depan.

```
System.out.println("Penelusuran mundur:");
backwardTraversal(third_3030);
```

2.3 Pratikum “InsertDLL_2511533030”

Langkah Kerja

1. Buat Class Baru
2. Buat Kode Berikut

```
package pekan6_2511533030;
public class InsertDLL_2511533030 {
    // menambahkan node di awal DLL
    static NodeDLL_2511533030 insertBegin(NodeDLL_2511533030 head_3030, int
data_3030) {
        // buat node baru
        NodeDLL_2511533030 new_node = new NodeDLL_2511533030(data_3030);
        // jadikan pointer next nya head
        new_node.next_3030 = head_3030;
        // jadikan pointer prev head ke new_node
    }
}
```

```

        if (head_3030 != null) {
            head_3030.prev_3030 = new_node;
        }
        return new_node;
    }
    // fungsi menambahkan node di akhir
    public static NodeDLL_2511533030 insertEnd(NodeDLL_2511533030 head_3030, int
newData_3030) {
        // buat node baru
        NodeDLL_2511533030 newNode = new NodeDLL_2511533030(newData_3030);
        // jika dll null jadikan head
        if (head_3030 == null) {
            head_3030 = newNode;
        }
        else {
            NodeDLL_2511533030 curr_3030 = head_3030;
            while (curr_3030.next_3030 != null) {
                curr_3030 = curr_3030.next_3030;
            }
            curr_3030.next_3030 = newNode;
            newNode.prev_3030 = curr_3030;
        }
        return head_3030;
    }
    // fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533030 insertAtPosition(NodeDLL_2511533030
head_3030, int pos_3030, int new_data_3030) {
        // buat node baru
        NodeDLL_2511533030 new_node = new NodeDLL_2511533030(new_data_3030);
        if (pos_3030 == 1) {
            new_node.next_3030 = head_3030;
            if (head_3030 != null) {
                head_3030.prev_3030 = new_node;
            }
            head_3030 = new_node;
            return head_3030;
        }
        NodeDLL_2511533030 curr_3030 = head_3030;
        for (int i_3030 = 1; i_3030 < pos_3030 - 1 && curr_3030 != null; ++i_3030)
        {
            curr_3030 = curr_3030.next_3030;
        }
        if (curr_3030 == null) {
            System.out.println("Posisi tidak ada");
            return head_3030;
        }
        new_node.prev_3030 = curr_3030;
        new_node.next_3030 = curr_3030.next_3030;
        curr_3030.next_3030 = new_node;
        if (new_node.next_3030 != null) {
            new_node.next_3030.prev_3030 = new_node;
        }
        return head_3030;
    }
    public static void printList(NodeDLL_2511533030 head_3030) {
        NodeDLL_2511533030 curr_3030 = head_3030;
        while (curr_3030 != null) {
            System.out.print(curr_3030.data_3030 + " <-> ");
            curr_3030 = curr_3030.next_3030;
        }
        System.out.println();
    }

    public static void main(String[] args) {

```

```

        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(2);
        head_3030.next_3030 = new NodeDLL_2511533030(3);
        head_3030.next_3030.prev_3030 = head_3030;
        head_3030.next_3030.next_3030 = new NodeDLL_2511533030(5);
        head_3030.next_3030.next_3030.prev_3030 = head_3030.next_3030;
        // cetak DLL awal
        System.out.print("DLL Awal: ");
        printList(head_3030);
        // tambah 1 di awal
        head_3030 = insertBegin(head_3030, 1);
        System.out.print(
            "simpul 1 ditambah di awal: ");
        printList(head_3030);
        // tambah 6 di akhir
        System.out.print(
            "simpul 6 ditambah di akhir:");
        int data_3030 = 6;
        head_3030 = insertEnd(head_3030, data_3030);
        printList(head_3030);
        // menambah node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4: ");
        int data2_3030 = 4;
        int pos_3030 = 4;
        head_3030 = insertAtPosition(head_3030, pos_3030, data2_3030);
        printList(head_3030);
    }
}

```

Gambar 2.3 kode program “InsertDLL_2511533030”

3. Didapatkan Output Sebagai Berikut

```

<terminated> InsertDLL_2511533030 [Java Application] C:\Users\AISYAH NAJMIATUL F\p2\pool\plugin
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir:1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Gambar 2.3 output program “InsertDLL_2511533030”

Penjelasan Kode

2.3.1 Deklarasi Class

- Bagian ini menunjukkan bahwa program berada di package **pekan6_2511533030** dan membuat kelas **InsertDLL_2511533030**

```

package pekan6_2511533030;
public class InsertDLL_2511533030 {

```

2.3.2 Fungsi Insert di Awal

- Membuat node baru.
- Pointer next dari node baru diarahkan ke head lama.
- Pointer prev dari head lama diarahkan ke node baru, kemudian node baru dijadikan head baru.

```

// menambahkan node di awal DLL
static NodeDLL_2511533030 insertBegin(NodeDLL_2511533030 head_3030, int
data_3030) {

```

```

        // buat node baru
        NodeDLL_2511533030 new_node = new NodeDLL_2511533030(data_3030);
        // jadikan pointer next nya head
        new_node.next_3030 = head_3030;
        // jadikan pointer prev head ke new_node
        if (head_3030 != null) {
            head_3030.prev_3030 = new_node;
        }
        return new_node;
    }
}

```

2.3.3 Fungsi Insert di Akhir

- Method ini digunakan untuk menambahkan node di bagian akhir Double Linked List
- Proses :
 - Membuat node baru
 - Menelusuri list sampai node terakhir
 - Menghubungkan node terakhir dengan node baru
- Perulangan digunakan untuk mencari node paling akhir.
- Menghubungkan node baru dengan node terakhir.

```

// fungsi menambahkan node di akhir
public static NodeDLL_2511533030 insertEnd(NodeDLL_2511533030 head_3030, int
newData_3030) {
    // buat node baru
    NodeDLL_2511533030 newNode = new NodeDLL_2511533030(newData_3030);
    // jika dll null jadikan head
    if (head_3030 == null) {
        head_3030 = newNode;
    }
    else {
        NodeDLL_2511533030 curr_3030 = head_3030;
        while (curr_3030.next_3030 != null) {
            curr_3030 = curr_3030.next_3030;
        }
        curr_3030.next_3030 = newNode;
        newNode.prev_3030 = curr_3030;
    }
    return head_3030;
}

```

2.3.4 Fungsi insert di Posisi Tertentu

- Proses :
 - Membuat node baru
 - Menelusuri node sampai posisi sebelum target
 - Mengatur pointer next dan prev
- Perulangan digunakan untuk mencari posisi sebelum tempat penyisipan node.
- Menghubungkan node baru dengan node sebelum dan sesudahnya.
- Node sebelumnya diarahkan ke node baru.

```

// fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511533030 insertAtPosition(NodeDLL_2511533030
head_3030, int pos_3030, int new_data_3030) {
    // buat node baru

```

```

NodeDLL_2511533030 new_node = new NodeDLL_2511533030(new_data_3030);
if (pos_3030 == 1) {
    new_node.next_3030 = head_3030;
    if (head_3030 != null) {
        head_3030.prev_3030 = new_node; }
    head_3030 = new_node;
    return head_3030; }
NodeDLL_2511533030 curr_3030 = head_3030;
for (int i_3030 = 1; i_3030 < pos_3030 - 1 && curr_3030 != null; ++i_3030)
{
    curr_3030 = curr_3030.next_3030; }
if (curr_3030 == null) {
    System.out.println("Posisi tidak ada");
    return head_3030; }
new_node.prev_3030 = curr_3030;
new_node.next_3030 = curr_3030.next_3030;
curr_3030.next_3030 = new_node;
if (new_node.next_3030 != null) {
    new_node.next_3030.prev_3030 = new_node; }
return head_3030;
}

```

2.3.5 Fungsi Menampilkan List

- Method **printList()** digunakan untuk mencetak seluruh isi Double Linked List dari depan ke belakang.
- Proseses :
 - Pointer bantu dimulai dari head.
 - Data dicetak satu per satu.
 - Pointer bergerak menggunakan next_3030.

```

public static void printList(NodeDLL_2511533030 head_3030) {
    NodeDLL_2511533030 curr_3030 = head_3030;
    while (curr_3030 != null) {
        System.out.print(curr_3030.data_3030 + " <-> ");
        curr_3030 = curr_3030.next_3030;
    }
    System.out.println();
}

```

2.3.6 Main Program

- Bagian **main()** digunakan untuk:
 - Membuat Double Linked List awal.
 - Menampilkan isi list.
 - Menambah node di awal.
 - Menambah node di akhir.
 - Menambah node pada posisi tertentu.

```

public static void main(String[] args) {
    // membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(2);
    head_3030.next_3030 = new NodeDLL_2511533030(3);
    head_3030.next_3030.prev_3030 = head_3030;
    head_3030.next_3030.next_3030 = new NodeDLL_2511533030(5);
    head_3030.next_3030.next_3030.prev_3030 = head_3030.next_3030;
    // cetak DLL awal
    System.out.print("DLL Awal: ");
}

```

```

        printList(head_3030);
        // tambah 1 di awal
        head_3030 = insertBegin(head_3030, 1);
        System.out.print(
            "simpul 1 ditambah di awal: ");
        printList(head_3030);
        // tambah 6 di akhir
        System.out.print(
            "simpul 6 ditambah di akhir:");
        int data_3030 = 6;
        head_3030 = insertEnd(head_3030, data_3030);
        printList(head_3030);
        // menambah node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4: ");
        int data2_3030 = 4;
        int pos_3030 = 4;
        head_3030 = insertAtPosition(head_3030, pos_3030, data2_3030);
        printList(head_3030);
    }
}

```

2.4 Pratikum “HapusDLL_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat Kode Berikut

```

package pekan6_2511533030;

public class HapusDLL_2511533030 {
    // fungsi menghapus node awal
    public static NodeDLL_2511533030 delHead(NodeDLL_2511533030 head_3030)
    {
        if (head_3030 == null) {
            return null;
        }
        NodeDLL_2511533030 temp = head_3030;
        head_3030 = head_3030.next_3030;
        if (head_3030 != null) {
            head_3030.prev_3030 = null;
        }
        return head_3030;
    }

    // fungsi menghapus di akhir
    public static NodeDLL_2511533030 delLast(NodeDLL_2511533030 head_3030)
    {
        if (head_3030 == null) {
            return null;
        }
        if (head_3030.next_3030 == null) {
            return null;
        }
        NodeDLL_2511533030 curr = head_3030;
        while (curr.next_3030 != null) {
            curr = curr.next_3030;
        }
        // update pointer previous node
        if (curr.prev_3030 != null) {

```

```

        curr.prev_3030.next_3030 = null;
    }
    return head_3030;
}
// fungsi menghapus node posisi tertentu
public static NodeDLL_2511533030 delPos(NodeDLL_2511533030 head_3030,
int pos_3030) {
    // jika DLL kosong
    if (head_3030 == null) {
        return head_3030;
    }
    NodeDLL_2511533030 curr = head_3030;
    // telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr != null && i < pos_3030; ++i) {
        curr = curr.next_3030;
    }
    // jika posisi tidak ditemukan
    if (curr == null) {
        return head_3030;
    }
    // Update pointer
    if (curr.prev_3030 != null) {
        curr.prev_3030.next_3030 = curr.next_3030;
    }
    if (curr.next_3030 != null) {
        curr.next_3030.prev_3030 = curr.prev_3030;
    }
    // jika yang dihapus head
    if (head_3030 == curr) {
        head_3030 = curr.next_3030;
    }
    return head_3030;
}

// fungsi mencetak DLL
public static void printList(NodeDLL_2511533030 head_3030) {
    NodeDLL_2511533030 curr = head_3030;
    while (curr != null) {
        System.out.print(curr.data_3030 + " ");
        curr = curr.next_3030;
    }
    System.out.println();
}

public static void main(String[] args) {

    // buat sebuah DLL
    NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(1);

    head_3030.next_3030 = new NodeDLL_2511533030(2);
    head_3030.next_3030.prev_3030 = head_3030;

    head_3030.next_3030.next_3030 = new NodeDLL_2511533030(3);
    head_3030.next_3030.next_3030.prev_3030 = head_3030.next_3030;

    head_3030.next_3030.next_3030.next_3030 = new
NodeDLL_2511533030(4);
    head_3030.next_3030.next_3030.next_3030.prev_3030 =
head_3030.next_3030.next_3030;

```

```

        head_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeDLL_2511533030(5);
        head_3030.next_3030.next_3030.next_3030.next_3030.prev_3030 =
            head_3030.next_3030.next_3030.next_3030;

        System.out.print("DLL Awal: ");
        printList(head_3030);

        System.out.print("Setelah head dihapus: ");
        head_3030 = delHead(head_3030);
        printList(head_3030);

        System.out.print("Setelah node terakhir dihapus: ");
        head_3030 = delLast(head_3030);
        printList(head_3030);

        System.out.print("menghapus node ke 2: ");
        head_3030 = delPos(head_3030, 2);

        printList(head_3030);
    }
}

```

Gambar 2.4 kode program “HapusDLL_2511533030”

3. Didapatkan Output Sebagai Berikut

```

<terminated> HapusDLL_2511533030 [Java Application] C:\Users\A
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
menghapus node ke 2: 2 4

```

Gambar 2.4 output program “HapusDLL_2511533030”

Penjelasan Kode :

2.4.1 Deklarasi Package dan Kelas

- Bagian ini menunjukkan bahwa program berada di package **pekan6_2511533030**.
- Kelas **HapusDLL_2511533030** digunakan untuk melakukan operasi penghapusan node pada Double Linked List

```

package pekan6_2511533030;
public class HapusDLL_2511533030 {

```

2.4.2 Fungsi Menghapus Node Awal

- Method **delHead()** digunakan untuk menghapus node pertama (head) pada Double Linked List.
- Proses :
 - Mengecek apakah list kosong.
 - Menyimpan head lama ke variabel sementara.
 - Head dipindahkan ke node berikutnya.
 - Pointer prev dari head baru diubah menjadi null.

```
// fungsi menghapus node awal
public static NodeDLL_2511533030 delHead(NodeDLL_2511533030 head_3030)
{
    if (head_3030 == null) {
        return null;
    }
    NodeDLL_2511533030 temp = head_3030;
    head_3030 = head_3030.next_3030;
    if (head_3030 != null) {
        head_3030.prev_3030 = null;
    }
    return head_3030;
}
```

2.4.3 Fungsi Menghapus Node Akhir

- Method **delLast()** digunakan untuk menghapus node terakhir pada Double Linked List.
- Proses :
 - Mengecek apakah list kosong.
 - Menelusuri node sampai node terakhir.
 - Pointer next dari node sebelumnya diubah menjadi null.

```
// fungsi menghapus di akhir
public static NodeDLL_2511533030 delLast(NodeDLL_2511533030 head_3030)
{
    if (head_3030 == null) {
        return null;
    }
    if (head_3030.next_3030 == null) {
        return null;
    }
    NodeDLL_2511533030 curr = head_3030;
    while (curr.next_3030 != null) {
        curr = curr.next_3030;
    }
    // update pointer previous node
    if (curr.prev_3030 != null) {
        curr.prev_3030.next_3030 = null;
    }
    return head_3030;
}
```

2.4.4 Fungsi Menghapus Node pada Posisi Tertentu

- Method **delPos()** digunakan untuk menghapus node pada posisi tertentu.
- Proses :
 - Mengecek apakah list kosong.
 - Menelusuri node sesuai posisi yang ingin dihapus.
 - Menghubungkan node sebelum dan sesudah node yang dihapus.
 - Jika node yang dihapus adalah head, maka head dipindahkan.

```
// fungsi menghapus node posisi tertentu
public static NodeDLL_2511533030 delPos(NodeDLL_2511533030 head_3030,
int pos_3030) {
    // jika DLL kosong
    if (head_3030 == null) {
```

```

        return head_3030;
    }
    NodeDLL_2511533030 curr = head_3030;
    // telusuri sampai ke node yang akan dihapus
    for (int i = 1; curr != null && i < pos_3030; ++i) {
        curr = curr.next_3030;
    }
    // jika posisi tidak ditemukan
    if (curr == null) {
        return head_3030;
    }
    // Update pointer
    if (curr.prev_3030 != null) {
        curr.prev_3030.next_3030 = curr.next_3030;
    }
    if (curr.next_3030 != null) {
        curr.next_3030.prev_3030 = curr.prev_3030;
    }
    // jika yang dihapus head
    if (head_3030 == curr) {
        head_3030 = curr.next_3030;
    }
    return head_3030;
}

```

2.4.5 Fungsi Menampilkan Double Linked List

- Method **printList()** digunakan untuk menampilkan isi Double Linked List dari depan ke belakang.
- Proses :
 - Pointer bantu dimulai dari head.
 - Data dicetak satu per satu.
 - Pointer bergerak menggunakan **next_3030**.

```

// fungsi mencetak DLL
public static void printList(NodeDLL_2511533030 head_3030) {
    NodeDLL_2511533030 curr = head_3030;
    while (curr != null) {
        System.out.print(curr.data_3030 + " ");
        curr = curr.next_3030;
    }
    System.out.println();
}

```

2.4.6 Main Program

- Membuat Double Linked List.
- Menampilkan isi awal list.
- Menghapus node pertama.
- Menghapus node terakhir.
- Menghapus node pada posisi tertentu.

```

public static void main(String[] args) {

    // buat sebuah DLL
    NodeDLL_2511533030 head_3030 = new NodeDLL_2511533030(1);

    head_3030.next_3030 = new NodeDLL_2511533030(2);
    head_3030.next_3030.prev_3030 = head_3030;

    head_3030.next_3030.next_3030 = new NodeDLL_2511533030(3);
}

```

```

        head_3030.next_3030.next_3030.prev_3030 = head_3030.next_3030;

        head_3030.next_3030.next_3030.next_3030 = new
NodeDLL_2511533030(4);
        head_3030.next_3030.next_3030.next_3030.prev_3030 =
head_3030.next_3030.next_3030;

        head_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeDLL_2511533030(5);
        head_3030.next_3030.next_3030.next_3030.next_3030.prev_3030 =
        head_3030.next_3030.next_3030.next_3030;

        System.out.print("DLL Awal: ");
        printList(head_3030);

        System.out.print("Setelah head dihapus: ");
        head_3030 = delHead(head_3030);
        printList(head_3030);

        System.out.print("Setelah node terakhir dihapus: ");
        head_3030 = delLast(head_3030);
        printList(head_3030);

        System.out.print("menghapus node ke 2: ");
        head_3030 = delPos(head_3030, 2);

        printList(head_3030);
    }
}

```

BAB III KESIMPULAN

3.1 Kesimpulan

Pada praktikum ini, telah dipelajari implementasi struktur data Double Linked List menggunakan bahasa Java. Double Linked List merupakan struktur data yang setiap node-nya memiliki dua pointer, yaitu next untuk menunjuk node berikutnya dan prev untuk menunjuk node sebelumnya. Dengan adanya dua pointer tersebut, data dapat ditelusuri dari dua arah, yaitu maju dan mundur.

Program yang dibuat meliputi pembuatan node, penelusuran data, penambahan node, dan penghapusan node. Penelusuran dilakukan menggunakan method traversal maju dan mundur. Penambahan node dapat dilakukan di awal, di akhir, maupun pada posisi tertentu. Selain itu, penghapusan node juga dapat dilakukan pada bagian awal, akhir, dan posisi tertentu pada Double Linked List.

Melalui praktikum ini, dapat dipahami bahwa penggunaan Double Linked List membuat pengelolaan data menjadi lebih fleksibel dibandingkan Single Linked List karena hubungan antar node dapat diakses dari dua arah. Praktikum ini juga melatih pemahaman mengenai penggunaan pointer serta logika dalam menghubungkan dan memanipulasi node pada struktur data dinamis.

TUGAS PRATIKUM

1. Membuat Class Lagu

```

package pekan6_2511533030;

public class Lagu_2511533030 {
    // atribut lagu
    private String judul_3030;
    private String penyanyi_3030;

    // pointer next dan prev
    Lagu_2511533030 next_3030;
    Lagu_2511533030 prev_3030;

    // constructor
    public Lagu_2511533030(String judul_3030, String penyanyi_3030) {
        this.judul_3030 = judul_3030;
        this.penyanyi_3030 = penyanyi_3030;
        this.next_3030 = null;
        this.prev_3030 = null;
    }

    // getter judul
    public String getJudul_3030() {
        return judul_3030;
    }

    // setter judul
    public void setJudul_3030(String judul_3030) {
        this.judul_3030 = judul_3030;
    }

    // getter penyanyi
    public String getPenyanyi_3030() {
        return penyanyi_3030;
    }

    // setter penyanyi
    public void setPenyanyi_3030(String penyanyi_3030) {
        this.penyanyi_3030 = penyanyi_3030;
    }
}

```

2. Membuat Class Musik

```

package pekan6_2511533030;
import java.util.Scanner;
public class Musik_2511533030 {
    // head dan tail playlist
    Lagu_2511533030 head_3030;
    Lagu_2511533030 tail_3030;

    // method tambah lagu di akhir
    public void tambahLagu_3030(String judul_3030, String penyanyi_3030) {
        Lagu_2511533030 laguBaru_3030 = new Lagu_2511533030(judul_3030,
        penyanyi_3030);

        // jika playlist kosong
        if (head_3030 == null) {
            head_3030 = laguBaru_3030;
            tail_3030 = laguBaru_3030;
        }
    }
}

```

```

    }
    // jika playlist sudah ada isi
    else {
        tail_3030.next_3030 = laguBaru_3030;
        laguBaru_3030.prev_3030 = tail_3030;
        tail_3030 = laguBaru_3030;
    }

    System.out.println("Lagu berhasil ditambahkan!");
}

// method hapus lagu awal
public void hapusLaguAwal_3030() {
    // jika playlist kosong
    if (head_3030 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    // jika hanya ada 1 lagu
    if (head_3030 == tail_3030) {
        head_3030 = null;
        tail_3030 = null;
    }
    // jika lebih dari 1 lagu
    else {
        head_3030 = head_3030.next_3030;
        head_3030.prev_3030 = null;
    }

    System.out.println("Lagu pertama berhasil dihapus!");
}

// method tampil maju
public void tampilMaju_3030() {
    // cek playlist kosong
    if (head_3030 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511533030 bantu_3030 = head_3030;

    System.out.println("\n=== Playlist Maju ===");

    while (bantu_3030 != null) {
        System.out.println("Judul      : " + bantu_3030.getJudul_3030());
        System.out.println("Penyanyi : " + bantu_3030.getPenyanyi_3030());
        System.out.println();

        bantu_3030 = bantu_3030.next_3030;
    }
}

// method tampil mundur
public void tampilMundur_3030() {
    // cek playlist kosong
    if (tail_3030 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
}

```

```

    Lagu_2511533030 bantu_3030 = tail_3030;

    System.out.println("\n=== Playlist Mundur ===");

    while (bantu_3030 != null) {
        System.out.println("Judul      : " + bantu_3030.getJudul_3030());
        System.out.println("Penyanyi : " + bantu_3030.getPenyanyi_3030());
        System.out.println();

        bantu_3030 = bantu_3030.prev_3030;
    }
}

// method cari lagu
public void cariLagu_3030(String judulCari_3030) {
    // cek playlist kosong
    if (head_3030 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511533030 bantu_3030 = head_3030;
    boolean ketemu_3030 = false;

    while (bantu_3030 != null) {
        // tidak case sensitive
        if (bantu_3030.getJudul_3030().equalsIgnoreCase(judulCari_3030)) {
            System.out.println("\nLagu ditemukan!");
            System.out.println("Judul      : " + bantu_3030.getJudul_3030());
            System.out.println("Penyanyi : " + bantu_3030.getPenyanyi_3030());
            ketemu_3030 = true;
            break;
        }

        bantu_3030 = bantu_3030.next_3030;
    }

    if (!ketemu_3030) {
        System.out.println("Lagu tidak ditemukan!");
    }
}

// main program
public static void main(String[] args) {
    Scanner input_3030 = new Scanner(System.in);
    Musik_2511533030 playlist_3030 = new Musik_2511533030();

    int pilihan_3030;

    do {
        System.out.println("\n=== Playlist Musik NIM: 2511533030 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");
        pilihan_3030 = input_3030.nextInt();
    }
}

```

```

        input_3030.nextLine();

        switch (pilihan_3030) {
            case 1:
                System.out.print("Judul Lagu : ");
                String judul_3030 = input_3030.nextLine();

                System.out.print("Penyanyi   : ");
                String penyanyi_3030 = input_3030.nextLine();

                playlist_3030.tambahLagu_3030(judul_3030, penyanyi_3030);
                break;

            case 2:
                playlist_3030.hapusLaguAwal_3030();
                break;

            case 3:
                playlist_3030.tampilMaju_3030();
                break;

            case 4:
                playlist_3030.tampilMundur_3030();
                break;

            case 5:
                System.out.print("Masukkan judul lagu yang dicari : ");
                String cari_3030 = input_3030.nextLine();

                playlist_3030.cariLagu_3030(cari_3030);
                break;

            case 6:
                System.out.println("Program selesai.");
                break;

            default:
                System.out.println("Pilihan tidak tersedia!");
        }

    } while (pilihan_3030 != 6);

    input_3030.close();
}
}

```

Output Program :

```

=== Playlist Musik NIM: 2511533030 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : Rosalia - Mio Cristo Piange Diamanti
Penyanyi   : Rosalia\
Lagu berhasil ditambahkan!

```

```
=== Playlist Musik NIM: 2511533030 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : Olivia Radingo - Drop Dead
Penyanyi : Olivia Rdirgo
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511533030 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : The Weekend - Cry For Me
Penyanyi : The Weekend
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511533030 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu pertama berhasil dihapus!
```

```
=== Playlist Musik NIM: 2511533030 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
```

```
=== Playlist Maju ===
Judul : Olivia Radingo - Drop Dead
Penyanyi : Olivia Rdirgo

Judul : The Weekend - Cry For Me
Penyanyi : The Weekend
```

```
=== Playlist Musik NIM: 2511533030 ===
```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

```
Pilihan : 4
```

```
=== Playlist Mundur ===
```

```
Judul      : The Weekend - Cry For Me
```

```
Penyanyi   : The Weekend
```

```
Judul      : Olivia Radingo - Drop Dead
```

```
Penyanyi   : Olivia Rdirgo
```

```
=== Playlist Musik NIM: 2511533030 ===
```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

```
Pilihan : 5
```

```
Masukkan judul lagu yang dicari : Olivia Radingo - Drop Dead
```

```
Lagu ditemukan!
```

```
Judul      : Olivia Radingo - Drop Dead
```

```
Penyanyi   : Olivia Rdirgo
```

```
=== Playlist Musik NIM: 2511533030 ===
```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

```
Pilihan : 6
```

```
Program selesai.
```